NoSQL Systems

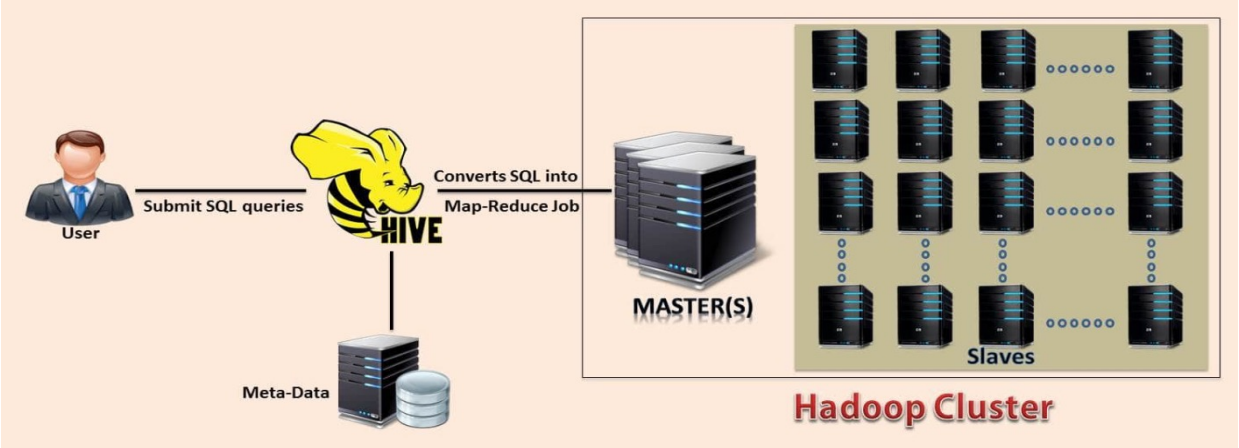
Apache Hive & HiveQL

"Data Warehouse and Query Language for Hadoop"

Vinu Venugopal
ScaDS.ai Lab, IIIT Bangalore

Hive and HiveQL

- Hive is an **ETL and data warehouse tool** on top of Hadoop ecosystem
- developed at **Facebook** in 2007-2008
- used for **querying and analyzing** large datasets stored in Hadoop file
- provides SQL-like declarative language, called **HiveQL**
- **Hive engine** compiles these **queries into Map-Reduce jobs** to be executed on Hadoop

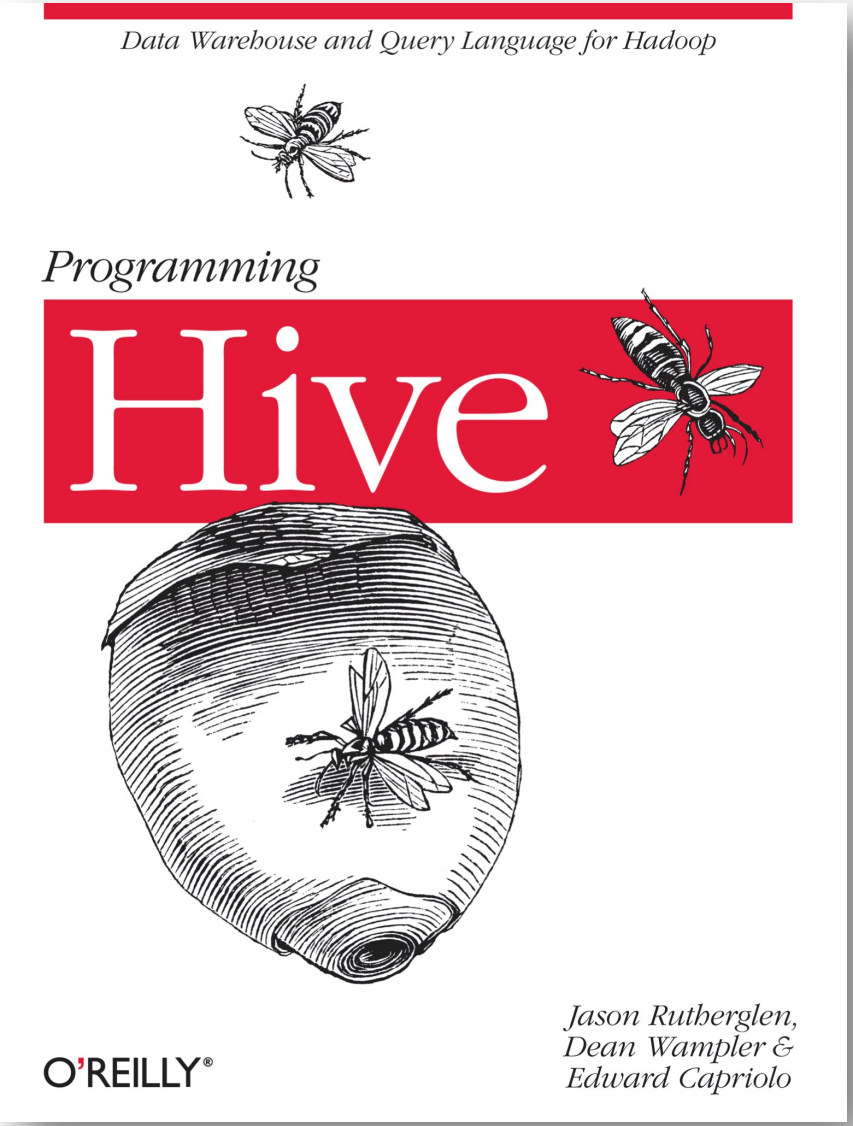


- constantly being developed and available from the Apache software foundation.

  (see: http://hive.apache.org 02 Oct 2025: release 4.2.0 available)

Hive Reference

"Apache Hive is a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale"



Hive Access Modes

- Hive shell
  - Interactive mode

```
% beeline -u jdbc:hive2://
hive> SHOW TABLES;
OK
Time taken: 18.457 seconds
```

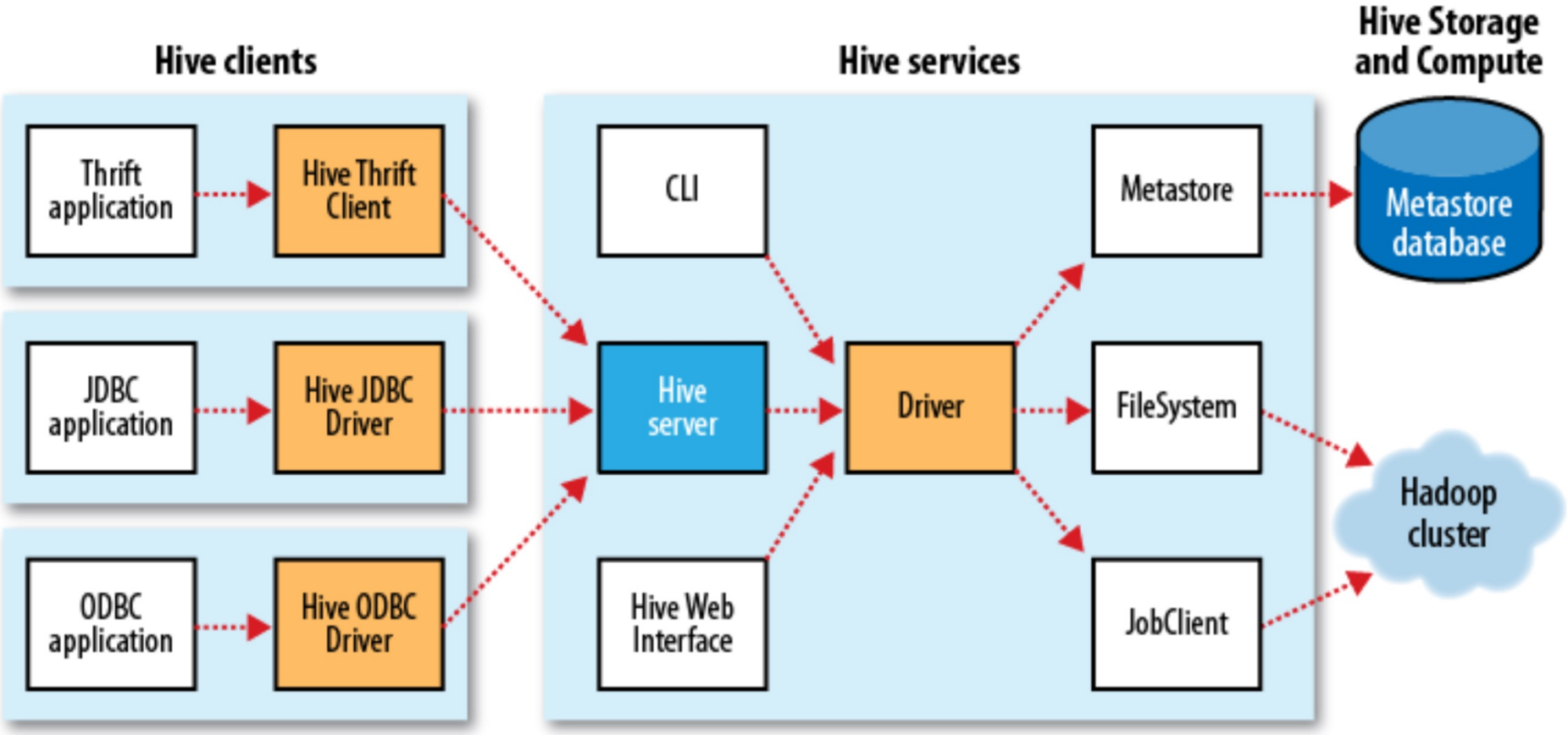
  - Non-Interactive mode (running HiveQL script)

```
% beeline -u jdbc:hive2://
hive> !run myscript.hiveql
```

- Other access modes:
  - Apache Thrift* clients
  - JDBC/ODBC clients

*a middleware layer to provide a **remote service interface** for executing queries and interacting with the *Hive Metastore*

Hive Architecture



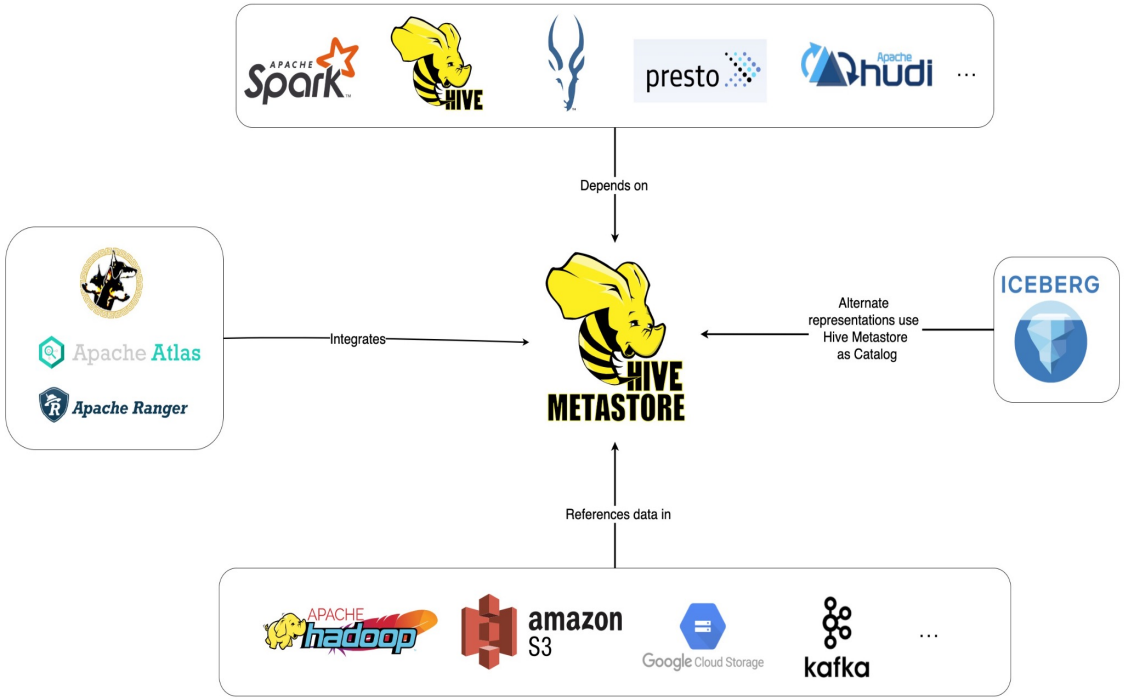
- The Hive Server is a single process running in a Java Virtual Machine (JVM).
- It communicates with the Hadoop FileSystem (HDFS), the Hadoop JobClient, and its own MetaStore (an actual DBMS).

Hive Architecture

Hive Metastore Server (HMS)

- A central repository of metadata for Hive tables and partitions in a relational database
- provides clients (including Hive, Impala and Spark) access to this information using the metastore service API.

The metastore is **not meant to interpret data**, only to **describe it**.



Hive Data Model

The basic data model of Hive, again, are nested relations (called "tables").

Elements of a relation may be of a *simple type* or of a *complex type*.

- 10 simple (atomic) data types:

  `TINYINT` (1-byte signed integer), `SMALLINT` (2-byte signed integer), `INT` (4-byte signed integer), `BIGINT` (8-byte signed integer), `FLOAT` (4-byte single-precision float), `DOUBLE` (8-byte double-precision float), `BOOLEAN` (true/false, 1-byte), `STRING` (char array), `BINARY` (byte array), `TIMESTAMP` (8-bytes)

- 3 complex types:

  `ARRAY`, `STRUCT`, `MAP` which are created via the built-in functions `array()`, `struct()`, `map()`.

Operators and Built-In Functions

HiveQL supports the usual set of SQL operators:

```
SELECT… FROM… WHERE… GROUP BY… HAVING… ORDER BY… LIMIT;
```

Built-in functions:

- Mathematical Functions e.g., round(DOUBLE a)
- Collection Functions e.g., size(Map<K.V>)
- Type Conversion Functions e.g., binary(stringlbinary)
- Date Functions e.g., to_date(string timestamp)
- Conditional Functions e.g., isnull(a)
- String Functions e.g., reverse(string A)
- Data Masking Functions fields e.g., mask_first_n(string str, int n)

```
hive> SHOW FUNCTIONS;
hive> DESCRIBE FUNCTION length;
```

Example: Table with Complex Types

```
hive> CREATE TABLE complex_table (
      col0 INT,
      col1 ARRAY<INT>,
      col2 MAP<STRING, INT>,
      col3 STRUCT<a:STRING, b:INT, c:DOUBLE>);
```

- Once after loading data..

```
hive> SELECT col0, col1[0], col2['b'], col3.c FROM
          complex_table;
```

Example: Table with Complex Types

```
hive> CREATE TABLE complex_table (
        col0 INT,
        col1 ARRAY<INT>,
        col2 MAP<STRING, INT>,
        col3 STRUCT<a:STRING, b:INT, c:DOUBLE>);

hive> SELECT col0, col1[0], col2['b'], col3.c FROM
          complex_table;
```

Delimiter	Description
\n	For text files, each line is a record, so the line feed character separates records.
^A ("control" A)	Separates all fields (columns). Written using the octal code \001 when explicitly specified in CREATE TABLE statements.
^B	Separate the elements in an ARRAY or STRUCT, or the key-value pairs in a MAP. Written using the octal code \002 when explicitly specified in CREATE TABLE statements.
^C	Separate the key from the corresponding value in MAP key-value pairs. Written using the octal code \003 when explicitly specified in CREATE TABLE statements.

Example: Table with Complex Types

```
hive> CREATE TABLE complex_table (
      col0 INT,
      col1 ARRAY<INT>,
      col2 MAP<STRING, INT>,
      col3 STRUCT<a:STRING, b:INT, c:DOUBLE>);

hive> SELECT col0, col1[0], col2['b'], col3.c FROM
         complex_table;
```

The above `CREATE TABLE` statement is actually short for:

```
CREATE TABLE …
 ROW FORMAT DELIMITED
 FIELDS TERMINATED BY '\001'
 COLLECTION ITEMS TERMINATED BY '\002'
 MAP KEYS TERMINATED BY '\003'
 LINES TERMINATED BY '\n'
 STORED AS TEXTFILE | [BINARY] RCFILE | [BINARY]  SEQUENCEFILE;
```

Example: Table with Complex Types

```
hive> CREATE TABLE complex_table (
      col0 INT,
      col1 ARRAY<INT>,
      col2 MAP<STRING, INT>,
      col3 STRUCT<a:STRING, b:INT, c:DOUBLE>);

hive> SELECT col0, col1[0], col2['b'], col3.c FROM
          complex_table;
```

Nested (i.e., complex) types are **serialized/de-serialized** by Hive using the built-in class:
`org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe`

Tables

Managed Table

External Table

Managed Tables

- **Internal tables → Hive controls the lifecycle of both the table and its data.**
- **Data is physically moved into HDFS** when loaded into the table.
- When loading data into a **managed table**, Hive **moves** the data to a default location:
  - hive.metastore.warehouse.dir (e.g., /user/hive/warehouse*)
  - This can be a distributed file system location like **HDFS**.

*We can **override this default location** using the LOCATION keyword when creating the table.

Tables

Managed Table

External Table

Managed Tables

- Stored at Hive's own internal location (another location in HDFS)

```
CREATE TABLE managed_tbl (field0 STRING)
LOCATION '/mytables/managed_tbl_table’;

LOAD DATA LOCAL INPATH '/myfiles/data.txt' INTO TABLE   managed_
tbl;
```

- `DROP TABLE managed_tbl;`
  - physically deletes all data in the Hive table!

Tables

Managed Table

External Table

Managed Tables

- Since Hive has complete control over **managed tables**,
  - **dropping the table also deletes the data** permanently from the warehouse directory.
- **Managed tables are not ideal for sharing data** across multiple tools like **Pig, Spark, or other external applications**, because Hive assumes full ownership of the data.
- If data needs to be **shared across multiple systems** or its lifecycle must be managed independently of Hive, an **external table should be used** instead of a managed table.

Tables

External Tables

- Data is NOT physically moved from HDFS (or S3, etc.)
- Merely keeps a reference to the file holding the data, together with the schema
- **External tables store only metadata** (schema and reference to data location), while the **actual data remains in its original HDFS (or other storage).**

- When using **CREATE EXTERNAL TABLE**, Hive **does not take ownership** of the data

```
CREATE EXTERNAL TABLE external_tbl (dummy STRING)
LOCATION '/hdfs/data';
```

Tables

External Tables

- **Dropping an external table does NOT delete the data**; it only removes the table's metadata from Hive.

- External tables are **ideal for sharing data** between different processing frameworks (e.g., **Hive, Spark, Pig**).

```
CREATE EXTERNAL TABLE external_tbl (dummy STRING)
LOCATION '/hdfs/data';

DROP TABLE external_tbl;
```

Schema Management

CREATE TABLE statement in Hive performs two basic steps:

1. Metastore update (ALWAYS happens)
   - True for both managed and external tables
   - Hive stores:
     - Schema (columns, types)
     - Table type (managed/external)
     - Data location
   - This is stored in the

2. Physical file/directory creation (only for managed table)
   - For managed table, Hive **creates a directory** in warehouse (e.g., /user/hive/warehouse/table_name/) Data is stored and managed by Hive

Schema Management

Schema-On-Write vs. Schema-On-Read

- Schema-On-Write: Data is **validated before storing.** Structure is enforced at write time. Used in traditional DBMS.

- Schema-On-Read: Data is stored **as-is.** Schema is applied **only when querying** Used in: Hive (both managed & external), Pig, Many big data systems

Note, bad data may go into HIVE. "INSERT" has some validation, LOAD DATA (file-based) has minimal or no validation, SELECT (query time) strict schema check would be preformed.

Partitions & Buckets

Partitions

- Horizontal slices of data
- Need? even a simple query in Hive reads the entire dataset

file2009

```
1,John,CS,2020
2,Mary,EE,2021
3,David,ME,2019
```

→ Parition-1

file2010

```
4,Alice,CS,2022
5,Bob,CE,2020
```

→ Parition-1

```
CREATE TABLE studentTab (
    id INT,
    name STRING,
    dept STRING,
    yoj INT
)
PARTITIONED BY (year STRING);
```

Load Data into Partitions

```
LOAD DATA LOCAL INPATH 'tab1/clientdata/2009/file2009'
INTO TABLE studentTab
PARTITION (year='2009');

LOAD DATA LOCAL INPATH 'tab1/clientdata/2010/file2010'
INTO TABLE studentTab
PARTITION (year='2010');
```

Partitions & Buckets

Partitions

- Horizontal slices of data
- Need? even a simple query in Hive reads the entire dataset

file2009

```
1,John,CS,2020
2,Mary,EE,2021
3,David,ME,2019
```

→ Parition-1

file2010

```
4,Alice,CS,2022
5,Bob,CE,2020
```

→ Parition-1

```
/studentTab/
    year=2009/
        file2009    (3 records)

    year=2010/
        file2010    (2 records)
```

Load Data into Partitions

```
LOAD DATA LOCAL INPATH 'tab1/clientdata/2009/file2009'
INTO TABLE studentTab
PARTITION (year='2009');

LOAD DATA LOCAL INPATH 'tab1/clientdata/2010/file2010'
INTO TABLE studentTab
PARTITION (year='2010');
```

Partitions & Buckets

Types of Partitioning:

- Static (we saw already!)
  - Specify partition key(s) and manually or "statically" move the data into each partition of the table
  - Can perform Static partition on Hive Manage table or external table
- Dynamic
  - Automatically load data into the partitions from a non-partitioned table or file
  - Takes more time in loading data compared to static partition
  - Can perform dynamic partition on Hive external table and managed table
  - Set property set hive.exec.dynamic.partition.mode=nonstrict*

*By default, Hive runs in strict mode for dynamic partitioning. Strict mode prevent accidental partition explosion.

Partitions & Buckets

Dynamic Partitioning E.g.

```
CREATE TABLE studentTab (
    id INT,
    name STRING,
    dept STRING,
    yoj INT
) PARTITIONED BY (year STRING);
```

file

```
1,John,CS,2020
2,Mary,EE,2021
3,David,ME,2019
4,Alice,CS,2022
5,Bob,CE,2020
```

insert

Parition-1

Parition-2

Parition-3

Storage:

```
/studentTab/
    year=2009/   (2 records)
    year=2010/   (2 records)
    year=2011/   (1 record)
```

Enable Dynamic Partitioning

```
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;
(SET hive.exec.max.dynamic.partitions=1000;      -- Increase if needed)
```

Note: Last column (yoj) is mapped to partition column year

Partitions & Buckets

Buckets

- partitions can be further subdivided into more manageable parts known as Buckets or Clusters.
- based on Hash function, which depends on the type of the bucketing column.
- *CLUSTERED BY* clause is used to divide the table into buckets.

```
CREATE TABLE bucketed_users (id INT, name STRING)
CLUSTERED BY (id) SORTED BY (id ASC) INTO 4 BUCKETS;
```

- partitions are stored in separate HDFS subdirectories, whereas buckets are kept in HDFS separate files (within each subdirectory).

Map-Side Join in Hive

One of the tables to be joined should be small enough to fit into main memory

```
SELECT /*+ MAPJOIN(s)*/ b.*, s.*
    FROM big_tbl b JOIN small_tbl s ON (b.id = s.id);
```

`/*+ MAPJOIN(things)*/` is a hint (similar to an optimizer hint, e.g., in Oracle) for loading the data of `small_tbl` into main memory, and to perform a Map-side merge join instead of the default Reduce-side join.

Map-Side Join in Hive

One of the tables to be joined should be small enough to fit into main memory

```
SELECT /*+ MAPJOIN(s)*/ b.*, s.*
    FROM big_tbl b JOIN small_tbl s ON (b.id = s.id);
```

`/*+ MAPJOIN(things)*/` is a hint (similar to an optimizer hint, e.g., in Oracle) for loading the data of `small_tbl` into main memory, and to perform a Map-side merge join instead of the default Reduce-side join.

This can be further optimized by bucketizing both the input tables:

```
CREATE TABLE big_tbl (id INT, name STRING)
    CLUSTERED BY (id) SORTED BY (id ASC) INTO 4 BUCKETS;

CREATE TABLE small_tbl (id INT, name STRING)
    CLUSTERED BY (id) SORTED BY (id ASC) INTO 4 BUCKETS;

SET hive.optimize.bucketmapjoin=true;
```

Note on Updates, Transactions and Indexes

…only have limited support in Hive

- Updates are supported via INSERT statements (thus via appending tuples to a table) only.
- Transactions are implemented via table- and partition-level locking.
- A simple form of an index is available via table partitions and buckets; more support for indexes is "in progress".

Updating and Appending Data

- `INSERT OVERWRITE` replaces the contents of the target table (or partition, resp.) with those of the source table (or subquery, see later slides).

- If `OVERWRITE` is omitted, records are appended.

```
INSERT OVERWRITE TABLE target
  SELECT col1, col2 FROM source;
```

- For partitioned tables, the target partition can be specified manually.

```
INSERT OVERWRITE TABLE target
   PARTITION (date='2013-04-25')
   SELECT col1, col2 FROM source;
```

- Just like in SQL, you can also create a new table directly from a subquery.

```
CREATE TABLE target AS SELECT col1, col2 FROM source;
```

Multi-Table Insert

Inserting the contents of one source table into multiple target tables!

```
FROM records
    INSERT OVERWRITE TABLE stations_by_year
      SELECT year, COUNT(DISTINCT station) GROUP BY year

   INSERT OVERWRITE TABLE records_by_year
      SELECT year, COUNT(1) GROUP BY year

    INSERT OVERWRITE TABLE good_records_by_year
      SELECT year, COUNT(1) WHERE temperature != 9999
        AND (quality = 0 OR quality = 1 OR quality = 4)
      GROUP BY year;
```

(→ Compare this feature to a multi-query execution in Pig!)

Sorting

Using standard "ORDER BY"

- Results would be totally sorted
- Sets the no. of reducers to one – inefficient for large datasets

Using nonstandard "SORT BY"

- Results would **not** be globally sorted
- Does not set the no. of reducers to one
- SORT BY produces a **sorted file per reducer**.
- Shard data using "DISTRIBUTE BY"

Input schema: `records(year int, station int, temperature float, quality int)`

```
hive> FROM records
        SELECT year, temperature
        DISTRIBUTE BY year
        SORT BY year ASC, temperature DESC;
```

```
SET mapreduce.job.reduces = 5;   OR
SET hive.exec.reducers.bytes.per.reducer = 256000000;  -- default ~256 MB
```

Joins

```
hive> SELECT * FROM sales;
Joe   2
Hank  4
Ali   0
Eve   3
Hank  2

hive> SELECT * FROM things;
2 Tie
4 Coat
3 Hat
1 Scarf
```

```
hive> SELECT sales.*, things.*
        FROM sales JOIN things ON
          (sales.id = things.id);
Joe   2 2 Tie
Hank  2 2 Tie
Eve   3 3 Hat
Hank  4 4 Coat
```

Note: the SQL-style abbreviation for an inner join `SELECT * FROM sales, things WHERE sales.id = things.id;` is not allowed!

- By default, translated into a Reduce-side join (unless specified otherwise) in MapReduce!
- The EXPLAIN command shows details about the join execution in MR.
- Hive supports only equi-joins.

```
EXPLAIN SELECT sales.*, things.* FROM sales JOIN things ON
        (sales.id = things.id);
```

Outer and Semi-Joins

```
hive> SELECT * FROM sales;
Joe  2
Hank 4
Ali  0
Eve  3
Hank 2

hive> SELECT * FROM things;
2 Tie
4 Coat
3 Hat
1 Scarf
```

```
hive> SELECT sales.*, things.*
         FROM sales LEFT OUTER JOIN
        things ON (sales.id =
        things.id);
```

```
hive> SELECT * FROM things LEFT
    SEMI JOIN sales ON (sales.id
= things.id);
2 Tie
3 Hat
4 Coat
```

- LEFT/RIGHT/FULL outer and semi-joins again follow the usual SQL semantics and are automatically translated into a Reduce-side join.
- Hive supports only outer and semi-joins with an equality condition.

Outer and Semi-Joins

```
hive> SELECT * FROM sales;
Joe  2
Hank 4
Ali  0
Eve  3
Hank 2

hive> SELECT * FROM things;
2 Tie
4 Coat
3 Hat
1 Scarf
```

```
hive> SELECT sales.*, things.*
         FROM sales LEFT OUTER JOIN
        things ON (sales.id =
        things.id);
Ali  0 NULL NULL
Joe  2 2 Tie
Hank 2 2 Tie
Eve  3 3 Hat
Hank 4 4 Coat
```

```
hive> SELECT * FROM things LEFT
    SEMI JOIN sales ON (sales.id
= things.id);
```

- LEFT/RIGHT/FULL outer and semi-joins again follow the usual SQL semantics and are automatically translated into a Reduce-side join.
- Hive supports only outer and semi-joins with an equality condition.

Outer and Semi-Joins

```
hive> SELECT * FROM sales;
Joe  2
Hank 4
Ali  0
Eve  3
Hank 2

hive> SELECT * FROM things;
2 Tie
4 Coat
3 Hat
1 Scarf
```

```
hive> SELECT sales.*, things.*
         FROM sales LEFT OUTER JOIN
        things ON (sales.id =
        things.id);
Ali  0 NULL NULL
Joe  2 2 Tie
Hank 2 2 Tie
Eve  3 3 Hat
Hank 4 4 Coat
```

```
hive> SELECT * FROM things LEFT
    SEMI JOIN sales ON (sales.id
= things.id);
2 Tie
3 Hat
4 Coat
```

- LEFT/RIGHT/FULL outer and semi-joins again follow the usual SQL semantics and are automatically translated into a Reduce-side join.
- Hive supports only outer and semi-joins with an equality condition.

Subqueries in HiveQL

- Hive has limited support for subqueries
- only permitting a subquery in the **FROM** clause of a **SELECT** statement.

```
SELECT station, year, AVG(max_temperature)
FROM (
  SELECT station, year, MAX(temperature) AS max_temperature
  FROM records
  WHERE temperature != 9999 AND (quality = 0 OR quality = 1)
  GROUP BY station, year
  DISTRIBUTE BY station, year
) max_temp
GROUP BY station, year
DISTRIBUTE BY station;
```

The above query is translated into two MapReduce jobs, one for each GROUP-BY clause.

Views in HiveQL

As in SQL, views are "virtual tables" which are not materialized. (Use CREATE TABLE if you want materialization instead.)

Input schema: records(year int, station int, temperature float, quality int)

```
CREATE VIEW valid_records AS
  SELECT * FROM records
  WHERE temperature != 9999 AND (quality = 0 OR quality = 1);
CREATE VIEW max_temperatures (station, year, max_temperature) AS
  SELECT station, year, MAX(temperature)
  FROM valid_records GROUP BY station, year
  DISTRIBUTE BY station, year;
SELECT station, year, AVG(max_temperature)
  FROM max_temperatures GROUP BY station, year
  DISTRIBUTE BY station;
```

The above series of view results in the same execution plan, using one MR job for each GROUP-BY clause!

User-Defined Functions in Hive

UDF's are implemented as **Java classes**

2 types of UDFs:

Basic UDFs: obtain a single row as input & produce a single row

Aggregate UDFs: obtain multiple rows as input iteratively & produce a single row

Example: Basic UDF

```
import org.apache.hadoop.hive.ql.exec.UDF;
…
public class Strip extends UDF {
  private Text result = new Text();
  public Text evaluate(Text str) {
    if (str == null) return null;
    result.set(StringUtils.strip(str.toString()));
    return result;
  }
  public Text evaluate(Text str, String stripChars) {
    if (str == null) return null;
    result.set(StringUtils.strip(str.toString(), stripChars));
    return result;
  }
}
```

Strip(' 00a11b00 ') => '00a11b00'

Strip('00a11b00', '01') => 'a11b' – remove chars in the 2nd argument from the prefix/suffix of 1st argument

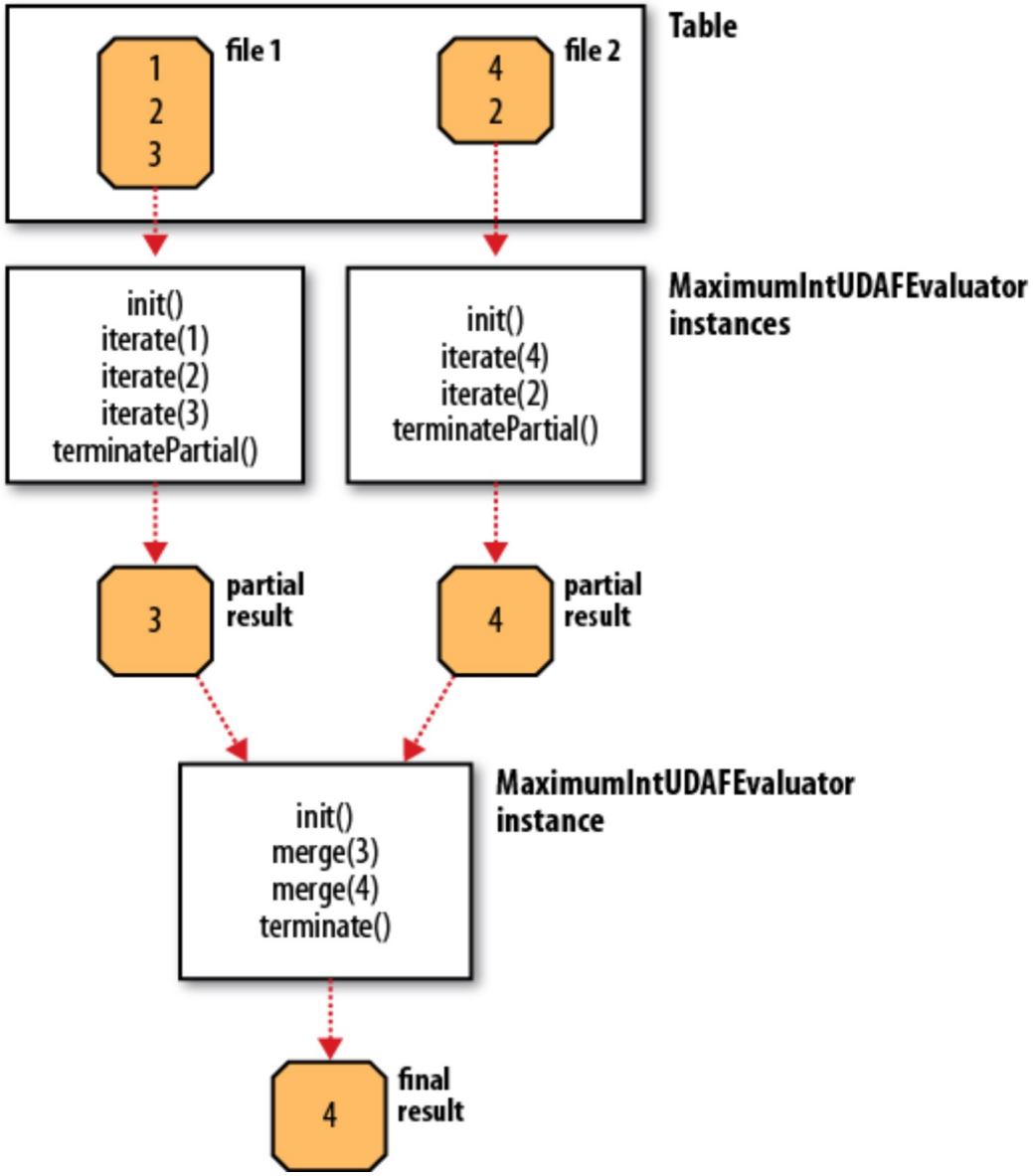
Example: Basic UDF

```
import org.apache.hadoop.hive.ql.exec.UDF;
…
public class Strip extends UDF {
  private Text result = new Text();
  public Text evaluate(Text str) {
    if (str == null) return null;
    result.set(StringUtils.strip(str.toString()));
    return result;
  }
  public Text evaluate(Text str, String stripChars) {
    if (str == null) return null;
    result.set(StringUtils.strip(str.toString(), stripChars));
    return result;
  }
}
```

```
ADD JAR ./stripexample.jar;

CREATE TEMPORARY FUNCTION strip AS 'Strip’;

SELECT strip(’00a11b00’, ‘01’) ;
```

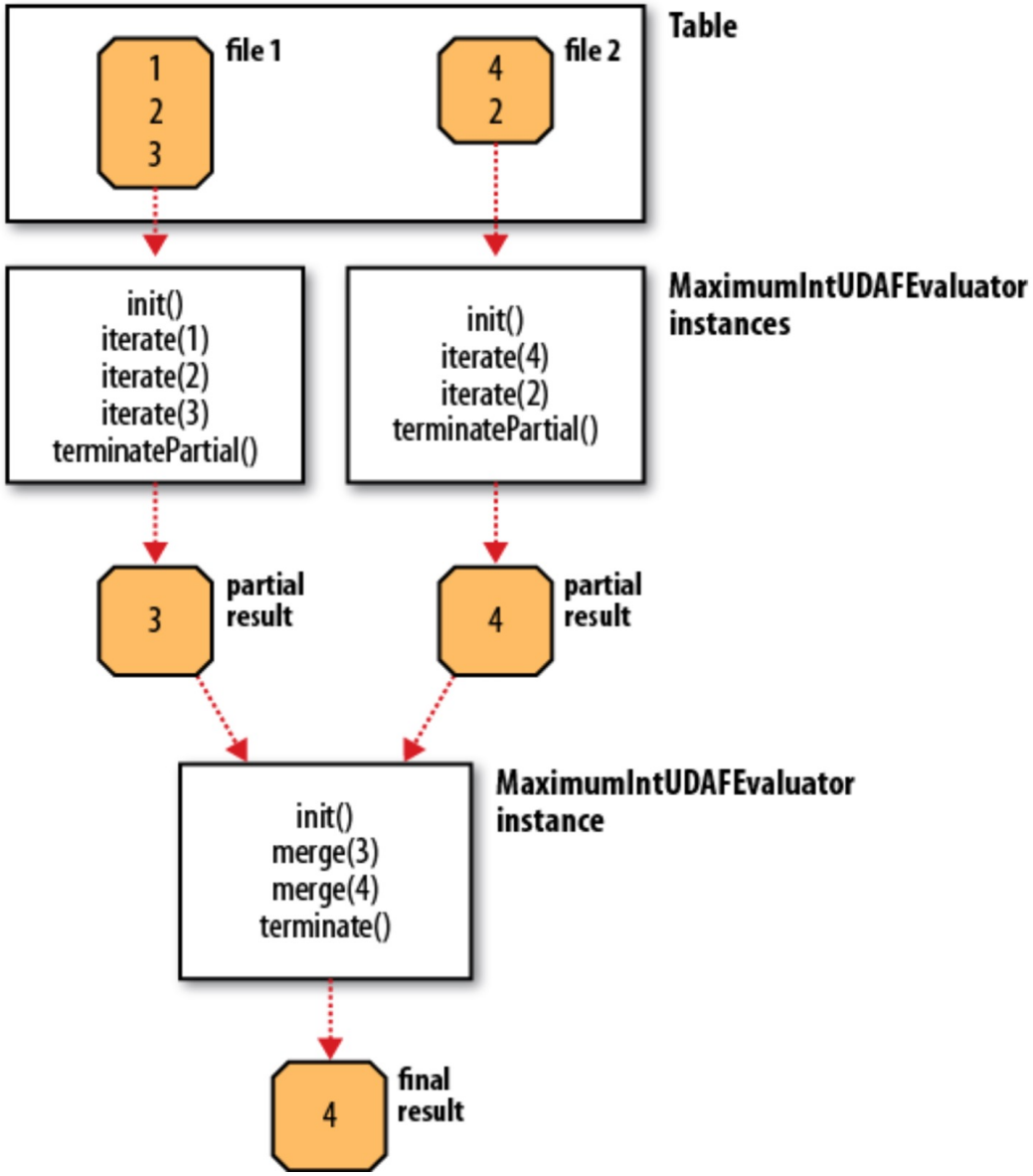
Example: Aggregate UDF (UDAF)

- Suppose we would like to (again) calculate the *Mean* of a set of input numbers in a distributed fashion.
- Further assume that the input number are split across different files.
- user-defined aggregate functions (UDAF's) in Hive may run in an arbitrary, distributed fashion inside a MapReduce job.



Example: Aggregate UDF (UDAF)

```
public class Mean extends UDAF {

 public static class PartialResult {
  double sum;
  long count;
 }

 public void init() {
 //Initialize the aggr buffer
 }

 public boolean iterate() {
 //Process a single row of i/p
 }

 public PartialResult terminatePartial() {
 //Return the partial aggr result
 }

 public boolean merge(PartialResult other) {
 //Merge another PartialResult
 }

 public DoubleWritable terminate() {
 //Returns the final aggr result
 }

}
```



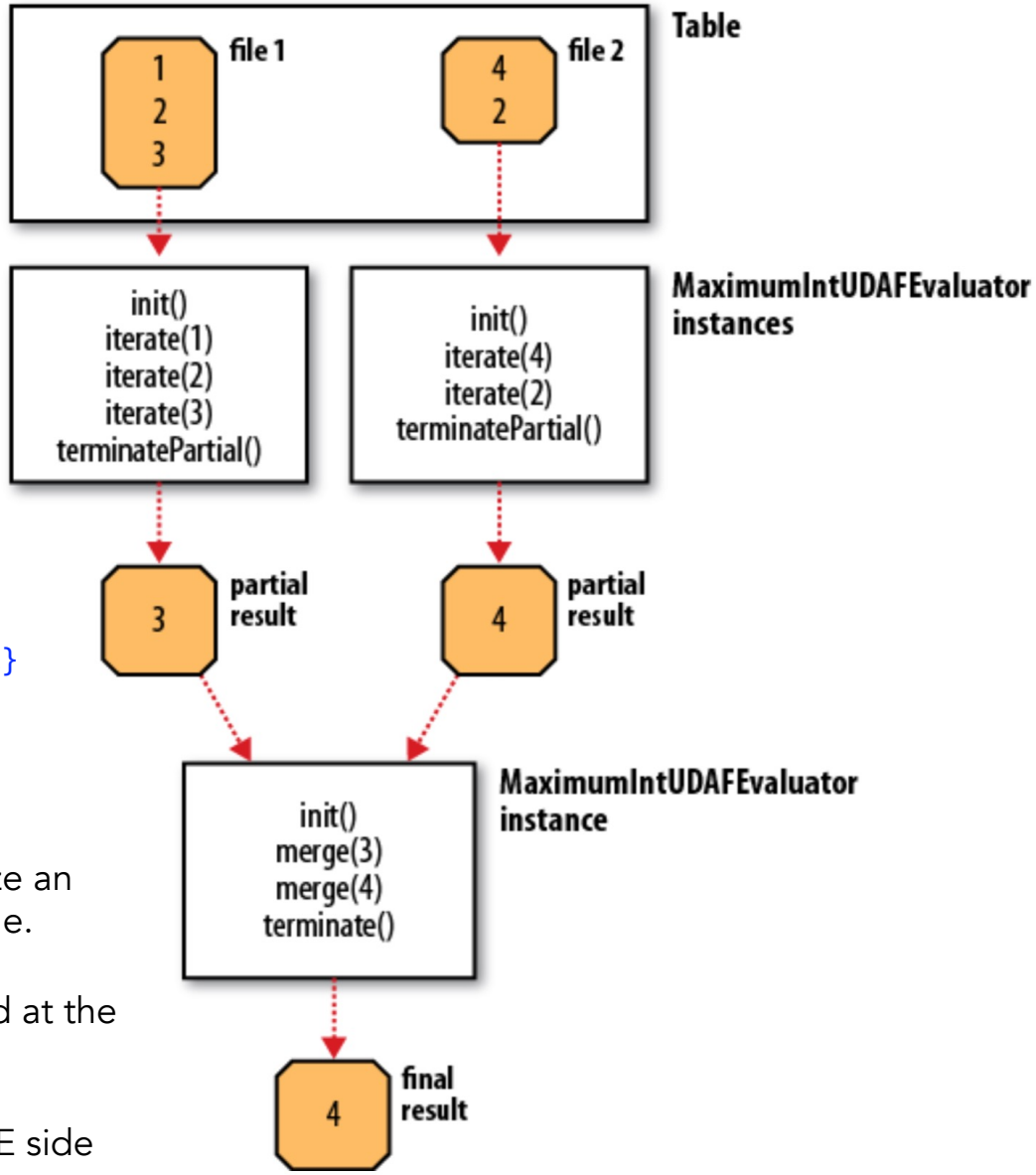
Example: Aggregate UDF (UDAF)

```
public class Mean extends UDAF {

 public static class PartialResult {
  double sum;
  long count;
 }

 public void init() {…}

 public boolean iterate() {…}

 public PartialResult terminatePartial() {…}

 public boolean merge(PartialResult other) {…}

 public DoubleWritable terminate() {…}

}
```

The init() method is called by the hive to initialize an instance of the UDAF evaluator class for each file.

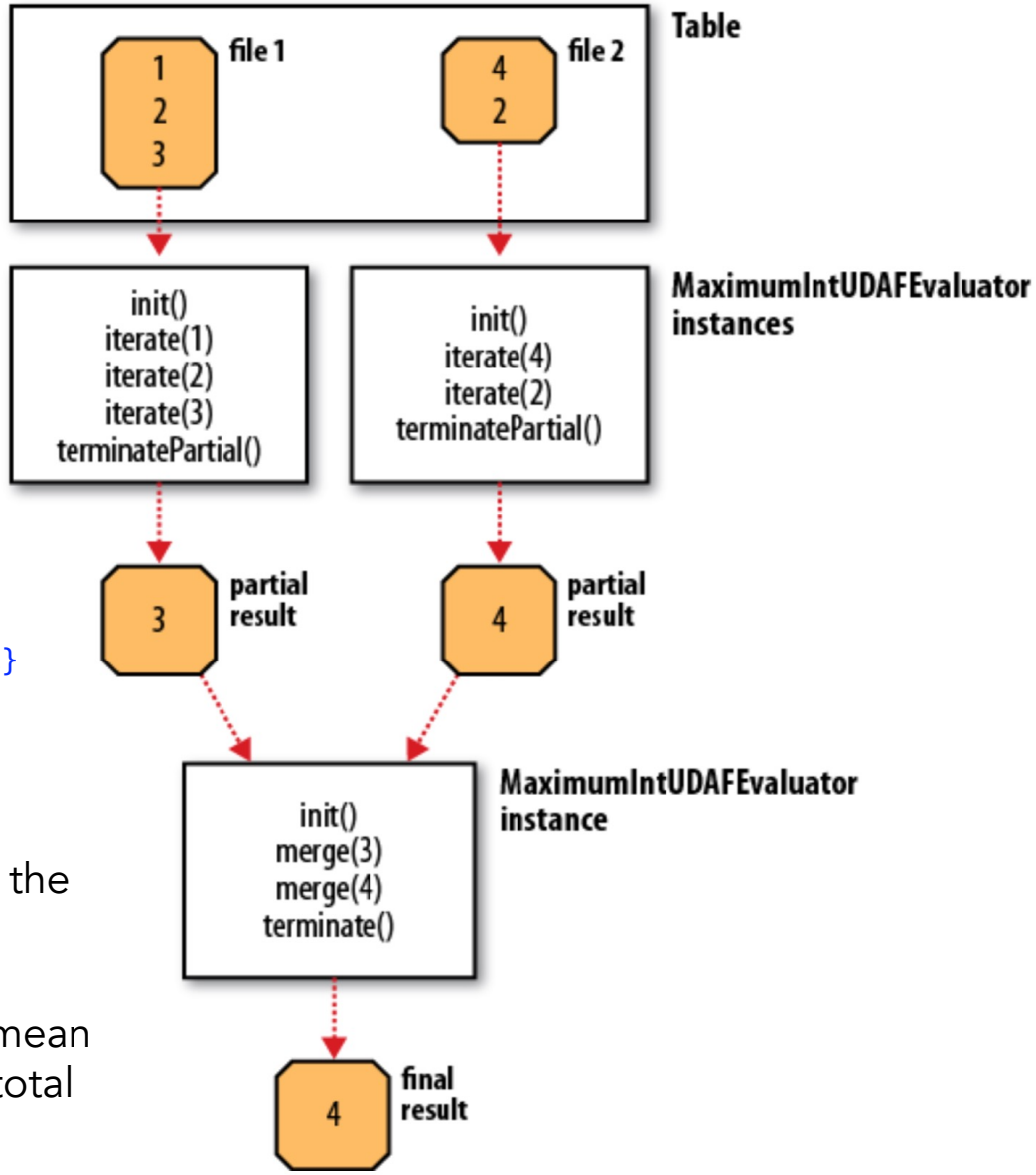
iterate() and terminatePartial() methods are used at the MAP side

terminate() and merge() are used at the REDUCE side



Example: Aggregate UDF (UDAF)

```
public class Mean extends UDAF {

 public static class PartialResult {
  double sum;
  long count;
 }

 public void init() {…}

 public boolean iterate() {…}

 public PartialResult terminatePartial() {…}

 public boolean merge(PartialResult other) {…}

 public DoubleWritable terminate() {…}

}
```

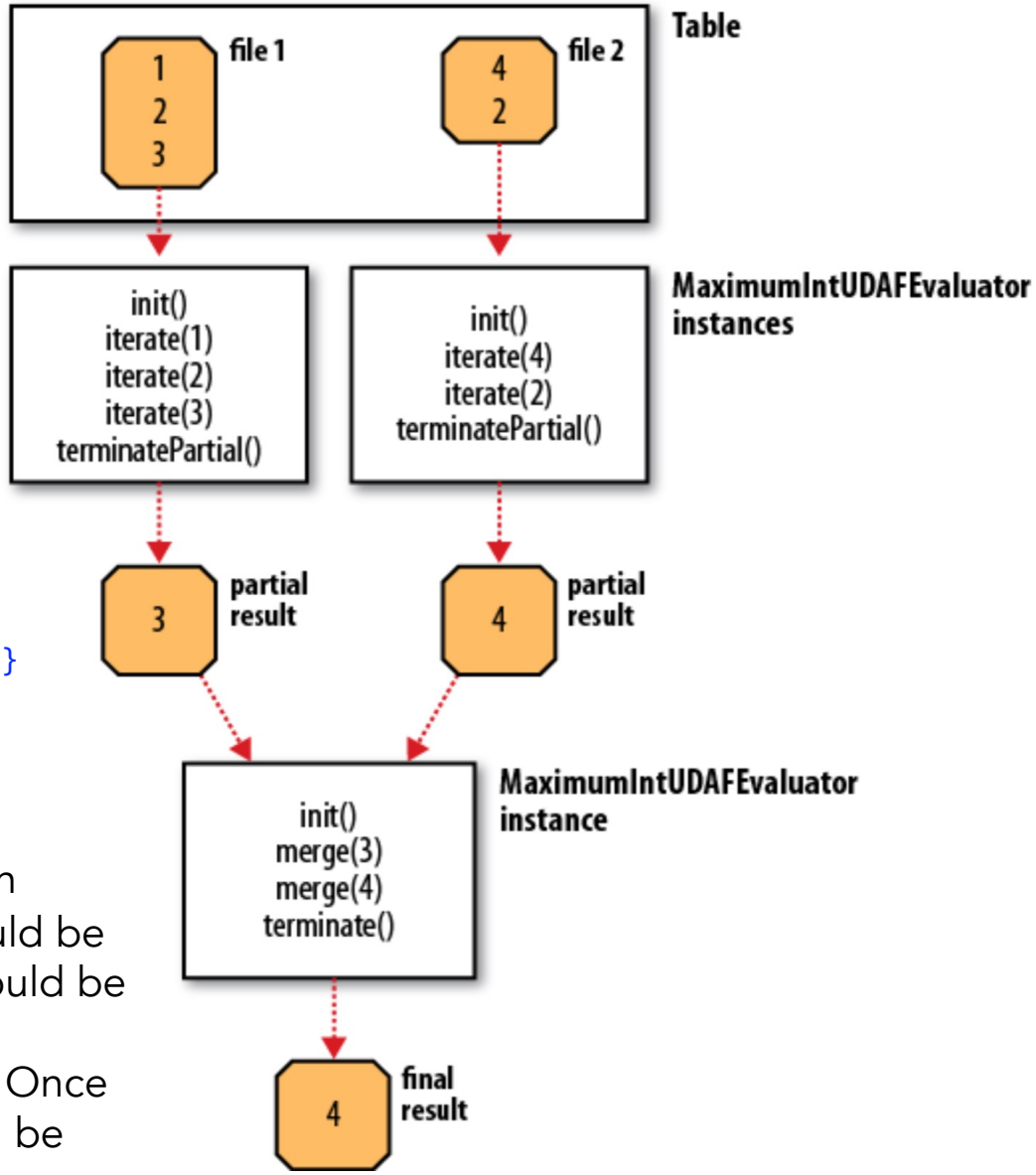


PartialResult class defines the structure of the partial results.

In the e.g., as the intention is to find the mean of all the values, map() should return the total sum and the count of numbers.

Example: Aggregate UDF (UDAF)

```
public class Mean extends UDAF {

 public static class PartialResult {
  double sum;
  long count;
 }

 public void init() {…}

 public boolean iterate() {…}

 public PartialResult terminatePartial() {…}

 public boolean merge(PartialResult other) {…}

 public DoubleWritable terminate() {…}

}
```



Iterate method will be executed over each numbers in the file, the partial results would be aggregated, and the terminatePartial() would be called once the inputsplit is finished.
Partial results will be returned to merge(). Once obtaining all partial results terminate() will be called to get final results.

Example: Aggregate UDF (UDAF)

See Mean.java
on Moodle!

Now calculate the *Mean* of a set of temperature values stored in the records table in a distributed way:

```
hive> SELECT mean(temperature) FROM records;
Total MapReduce jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time:1
…

23.4
```